Aufgabengebiet I IT

Montag, 9. Februar 2026 09:59

Aufgabe:

Simuliert selbst ein Sprachmodell.

Gegeben ist folgender Satzanfang:

„Ein Server ist ein Gerät, das…“

Schreibt nun **Wort für Wort** weiter.
Dabei darf jedes neue Wort nur das sein, was **am wahrscheinlichsten** erscheint.

Beispiel:

- verarbeitet
- Daten
- und
- stellt
- Dienste
- bereit

Reflexionsfragen:

1. Warum habt ihr genau diese Wörter gewählt?
2. Hattet ihr „Verständnis“ oder habt ihr Muster genutzt?
3. Was wäre eine unwahrscheinliche, aber grammatikalisch mögliche Fortsetzung?

Teil 2 – Analyse eines KI-Ergebnisses (Einzel- oder Partnerarbeit)

Verwendet ein KI-Tool und gebt folgenden Prompt ein:

„Erkläre einem Kunden, warum ein lokaler Server sicherer ist als eine Cloud-Lösung.“

Arbeitsauftrag:

Analysiert die Antwort unter folgenden Gesichtspunkten:

1. Sind alle Aussagen technisch korrekt?
2. Werden Sicherheitsaspekte differenziert betrachtet?
3. Fehlen wichtige Argumente?
4. Klingt die Antwort überzeugend – auch ohne Belege?
5. Könnte hier eine Halluzination enthalten sein?

Teil 3 – Theorieverständnis sichern

Beantwortet schriftlich:

1. Wie funktionieren Sprachmodelle technisch vereinfacht?

Erklärt in eigenen Worten:

- Was sind Trainingsdaten?

- Was bedeutet Wahrscheinlichkeitsprinzip?
- Warum „versteht“ eine KI nicht wirklich?

2. Erklärt den Unterschied zwischen:

- Mustererkennung
- menschlichem Verständnis

Bezieht euch dabei auf Programmierung:

Ist Autocomplete in einer IDE „intelligent“?

Teil 4 – Stärken- und Schwächenanalyse (Anwendungsbezug)

Füllt folgende Tabelle aus:

Bereich	Stärke von KI Risiko/Schwäche
Codegenerierung	
Dokumentation	
Sicherheitsbewertung	
Architekturentscheidungen	

Teil 5 – Transfer in den Entwickleralltag

Stellt euch vor:

Ein Azubi übernimmt ungeprüft KI-generierten Code in ein Kundenprojekt.

Diskussionsfragen:

- Welche Risiken entstehen?
- Wer trägt die Verantwortung?
- Wie sollte man KI im Entwicklungsprozess richtig einsetzen?

Bonus (für leistungsstärkere Gruppen)

Lasst euch von der KI eine nicht existierende Java-Funktion erklären (z. B. „java.security.UltraEncryptor“).

- Erfindet die KI eine Erklärung?
- Wie überzeugend wirkt sie?
- Was sagt das über Halluzinationen aus?

Teil 1
Ein Server ist ein Gerät, das Daten verarbeitet und Dienste für Clients bereitstellt.
Teil 1.

1. Eine kurze Erklärung, was Server überhaupt sind - sowie deren wichtigsten Aufgaben. Dabei habe ich es so kurz und kompakt wie nur möglich zusammengefasst.
2. Ja ich habe ein Muster angewandt, da ich selber schon viele Ausgaben der KI gesehen habe.
3. Ein Server ist ein Gerät, das Daten verarbeitet und Dienste für Clients bereitstellt. Server werden ebenfalls als Storage für große Datenmengen benutzt. Server benutzt jeder, der im Internet servt.

Teil 2
Arbeitsauftrag:

Analysiert die Antwort unter folgenden Gesichtspunkten:

1. Sind alle Aussagen technisch korrekt? Ja
2. Werden Sicherheitsaspekte differenziert betrachtet? Ja
3. Fehlen wichtige Argumente? Bestimmt, ich wüsste aber keine
4. Klingt die Antwort überzeugend - auch ohne Belege? Ja
5. Könnte hier eine Halluzination enthalten sein? Ja

Teil 3

Ein Sprachmodell liest sehr viel Text und lernt daraus Muster. Beim Antworten „rät“ es dann das **wahrscheinlich nächste Wort** (genauer: das nächste Token) und setzt so Satz für Satz zusammen.

Was sind Trainingsdaten?

- **Trainingsdaten** sind riesige Mengen an Texten (z. B. Webseiten, Bücher, Foren, Dokus, Code).
- Daraus lernt das Modell: **Welche Wörter kommen oft zusammen?** Welche Satzstrukturen sind typisch?
- Wichtig: In den Daten können auch **Fehler, Vorurteile oder veraltete Infos** stecken.

Was bedeutet Wahrscheinlichkeitsprinzip?

- Das Modell hat keine festen Regeln wie ein Taschenrechner.
- Es berechnet: **Wie wahrscheinlich ist Wort A/B/C als nächstes?**
- Beispiel: „Ich trinke gerne …“ → „Wasser“ ist wahrscheinlicher als „Schraubenzieher“.

Warum „versteht“ eine KI nicht wirklich?

- Sie hat **keine echten Erfahrungen**, keine Gefühle, keinen „gesunden Menschenverstand“.
- Sie prüft nicht automatisch, ob etwas **wahr** ist.
- Sie erzeugt Text, der **plausibel klingt**, aber manchmal trotzdem falsch ist (Halluzinationen).

2) Unterschied zwischen Mustererkennung und menschlichem Verständnis

Mustererkennung (KI):

- Erkennt Wiederholungen und typische Kombinationen („Patterns“).
- Kann sehr gut Texte nachahmen und Vorschläge machen.
- Aber: weiß nicht wirklich, **was** etwas bedeutet.

Menschliches Verständnis:

- Bedeutungen, Ziele, Kontext, Erfahrung, Logik, echte Absicht.
- Kann Dinge bewerten wie: „Ist das sinnvoll? Ist das gefährlich? Passt das zur Situation?“

Bezug auf Programmierung: Ist Autocomplete in einer IDE „intelligent“?

- **Autocomplete** ist meist **nicht wirklich intelligent**, sondern:
  - ergänzt Namen, Methoden, Variablen anhand von Syntax/Typen/Projektstruktur.
- Moderne „KI-Autocomplete“ (z. B. Copilot) wirkt schlauer, weil es **Muster aus Code** nutzt.
- Trotzdem: Es kann **falsch**, **unsicher** oder **nicht passend** sein → muss geprüft werden.

Teil 4 – Stärken- und Schwächenanalyse (Anwendungsbezug)

Bereich	Stärke von KI	Risiko/Schwäche
Codegenerierung	Schnell Codevorschläge, Boilerplate, Beispiele, einfache Funktionen	Kann Bugs einbauen, falsche Annahmen treffen, unsicheren Code erzeugen, passt evtl. nicht zum Projektstil
Dokumentation	Formuliert gut, fasst Code zusammen, schreibt README/Kommentare schnell	Kann Dinge erfinden oder falsch erklären, wird evtl. zu allgemein oder ungenau
Sicherheitsbewertung	Findet typische Schwachstellen-Muster (z. B. SQL Injection-Hinweise), gibt Checklisten	Keine Garantie, übersieht Probleme, kennt nicht alle Projekt-Details, kann falsche Sicherheit geben
Architekturentscheidungen	Liefert Ideen, vergleicht Ansätze, nennt Vor-/Nachteile, hilft beim Brainstorming	Kennt eure echten Anforderungen oft nicht, kann „schön klingende“ aber unpassende Architektur empfehlen

Teil 5 – Transfer in den Entwicklerallltag

Situation: Ein Azubi übernimmt ungeprüft KI-generierten Code in ein Kundenprojekt.

Welche Risiken entstehen?

- **Bugs & Fehlfunktionen:** Der Code läuft vielleicht, aber macht nicht das Richtige.
- **Sicherheitslücken:** z. B. unsichere Eingaben, fehlende Auth, Datenlecks.
- **Qualitätsprobleme:** schlechter Stil, keine Tests, schwer wartbar.
- **Lizenz/Compliance-Risiko:** Code könnte zu nah an fremdem Code sein (selten, aber möglich).
- **Kosten & Ruf:** Kunde hat Fehler → Nacharbeit, Vertrauensverlust.

Wer trägt die Verantwortung?

- **Nicht die KI.**
- Verantwortung liegt bei:
  - der Person, die den Code einbaut (Azubi)
  - und besonders beim Team/Unternehmen, weil es **Review- und Freigabeprozesse** geben muss
    (in der Praxis: **Betreuer/Lead/Code-Owner** muss prüfen).

Wie sollte man KI richtig einsetzen?

- KI als **Assistent**, nicht als „Autopilot“:
  1. **Verstehen**: Erst prüfen, ob man den Code wirklich versteht.
  2. **Review**: Code-Review durch erfahrene Person (4-Augen-Prinzip).
  3. **Tests**: Unit-Tests/Integrationstests schreiben und ausführen.
  4. **Security-Check**: Eingaben validieren, Secrets vermeiden, OWASP-Grundregeln beachten.
  5. **Projektregeln einhalten**: Coding-Standards, Architektur, Logging, Error-Handling.
  6. **Dokumentieren**: Was wurde übernommen, warum, welche Annahmen?

Bonus:

Bonus (für leistungsstärkere Gruppen)

Lasst euch von der KI eine nicht existierende Java-Funktion erklären (z. B. „java.security.UltraEncryptor“).

- Erfindet die KI eine Erklärung?
- Wie überzeugend wirkt sie?
- Was sagt das über Halluzinationen aus?

java.security.UltraEncryptor ist **keine** Klasse/Funktion aus der offiziellen Java-Standardbibliothek. In den Oracle-Javadocs für das Paket java.security (Java SE 8 bis 25) taucht sie nicht auf.
Wichtig dabei: **Eigene Klassen in java.*-Paketen** (z. B. java.security.*) darf man normalerweise **nicht** erstellen/landen lassen – das führt typischerweise zu SecurityException: Prohibited package name

Was bedeutet das praktisch?

Wenn du irgendwo java.security.UltraEncryptor siehst, ist sehr wahrscheinlich eins davon der Fall:

- **Tippfehler / Verwechslung** (z. B. mit javax.crypto.Cipher oder java.security.MessageDigest)
- **Obfuskierter/dekompilierter Code** oder **Fake-Beispiel** aus einem Tutorial

- **Eine externe Library**, die so heißt *oder* ein eigenes Package falsch gewählt hat (was dann zur oben genannten Exception führen kann)

Wirkt überzeugend